Algoritmi cu mesaje de sondaj cu ecou

- Ideea :
  - un **Inițiator** colectează informații despre topologia locală a tuturor celorlalte noduri, determină topologia întregii rețele și o difuzează nodurilor.
- **Cazul topologiilor arbore ⇒ sursa (initiatorul) este rădacina arborelui**
  - Două faze :
    - mesajele de sondaj se propagă de la inițiator spre frunze
    - mesajele de ecou *(care conțin topologii parțiale)* se propagă de la frunze spre inițiator.

Algoritmi cu mesaje de sondaj cu ecou (2)

```
const int inițiator = indexul nodului
inițiator;

typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondaj[1:N](int transm);
chan ecou[1:M]  (tip_top top);
chan ecou_final (tip_top top);

process Nod[p=1 to N]{
   tip_leg leg = vecinii_lui_p;
   tip_top top = ([N * N] FALSE);
   int părinte = N + 1;
   tip_top top_nou;

   top[p, 1:n] = leg;
```

Algoritmi cu mesaje de sondaj cu ecou (3)

```
if (p <> inițiator)
   receive sondaj[p](părinte);

/* transmite sondaje celorlalte noduri vecine, copiii
lui p */
for [q = 1 to N st (leg[q] AND (q <> părinte))]
   send sondaj[q](p);

/* primește ecourile și fă reuniunea lor */
for [q = 1 to N st (leg[q] AND (q <> părinte))] {
   receive ecou[p](top_nou);
   top = top OR top_nou;
}

if (p <> inițiator)
   send ecou[părinte](top);
}
```

Algoritmi cu mesaje de sondaj cu ecou (4)

- **Cazul general**
  - după ce primeşte un prim mesaj de sondaj, un nod îl propagă tuturor celorlalţi vecini
  - nodul aşteaptă ecouri de la vecini
  - **există cicluri** ⟹ nodul poate primi și sondaje
    - topologia locală va fi transmisă doar ca ecou la primul sondaj
    - pentru restul sondajelor se transmite o topologie nulă
    - sondajele și ecourile trebuie recepţionate pe acelaşi canal

Algoritmi cu mesaje de sondaj cu ecou (5)

```
/* index inițiator */
const inițiator = indexul nodului inițiator;

enum fel{sondă, ecou};
typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondă-ecou[1:N](fel tip_mesaj, int transm, tip_top
top);
chan ecou_final(tip_top top);

process Nod[p=1 to N]{
   tip_leg leg = vecinii_lui_p;
   tip_top top = ([N * N] FALSE);
   int părinte = n + 1;
   tip_top top_nou;
```

Algoritmi cu mesaje de sondaj cu ecou (6)

```
   /* actualizează vecinii lui p */
   top[p, 1:N] = leg;

   fel k; int transm;
   int nr_ecouri = 0;

   if (p <> inițiator)
      receive sondă-ecou[p](k, părinte, top_nou);

   /* transmite sondaje celorlalte noduri vecine, copiii lui p
*/
   for [q = 1 to N st (leg[q] AND (q <> părinte))] {
      send sondă-ecou[q](sondă, p, 0);  /* topologie nulă */
      nr_ecouri = nr_ecouri + 1;
   }
```

Algoritmi cu mesaje de sondaj cu ecou (7)

```
   /* primește ecourile și fă reuniunea lor, sau sonde și
ignoră */
   while (nr_ecouri > 0) {
      receive sondă-ecou[p](k, transm, top_nou);

      if (k == sondă)
         send sondă-ecou[transm](ecou, p, 0);
      else if (k == ecou) {
         top = top OR top_nou;
         nr_ecouri = nr_ecouri - 1;
      }
   }

   if (p <> inițiator)
      send sondă-ecou[părinte](ecou, p, top);
```